week_11

1. scheme， 一种新的编程语言

```
No return in Scheme; the value of a call expression is the value of the last body
expression of the procedure
```

```
>>> def sum_squares(x, y):
...     return x * x + y * y
```

```
scm> (define (sum-squares x y)
       (+ (* x x) (* y y)))
```

```
Instead of multiple return statements, Scheme uses nested conditional expressions.
```

```
>>> def fib(n):
...     if n == 0 or n == 1:
...         return n
...     else:
...         return fib(n - 2) + fib(n - 1)
```

```
scm> (define (fib n)
       (if (or (= n 0) (= n 1))
           n
           (+ (fib (- n 2)) (fib (- n 1)))))
```

对比维度	结论
是否有直接关系?	没有直接继承关系，但都属于高级语言
是否可以互相替代?	在大多数实际项目中不可替代，但都可以用于教学
哪个更适合入门?	Python 更适合编程初学者
哪个更适合理解编程本质?	Scheme 更适合理解语言设计、递归、函数式编程
哪个更实用?	Python 更实用，拥有庞大的生态系统和广泛应用
哪个更"纯粹"?	Scheme 更接近编程语言理论，Python 更注重实用性

Python vs Scheme: Iteration

Scheme has no for/while statements, so recursion is required to iterate.

```
>>> def sum_first_n(n):
...     return sum(range(1, n + 1))
...
>>> def sum_first_n(n):
...     total = 0
...     for k in range(1, n + 1):
...         total += k
...     return total
...
>>> def sum_first_n(n):
...     k = 1
...     total = 0
...     while k <= n:
...         k, total = k + 1, total + k
...     return total
...
>>> sum_first_n(5)
15
```

```
scm> (define (sum-first-n n)
       (define (f k total)
         (if (> k n)
             total
             (f (+ k 1) (+ total k))))
       (f 1 0))
sum-first-n
scm> (sum-first-n 5)
15
```

这个例子给我看笑了，有被无语到

```
(define (a-plus-abs-b a b)
  ((if (< b 0) - +) a b))
```

这个例子更是高手，通过函数直接可以传递正负号

2. scheme中lambda函数的定义

Lambda Expressions

Lambda expressions evaluate to anonymous procedures

```
(lambda (<formal-parameters>) <body>)
```

Two equivalent expressions:

```
(define (plus4 x) (+ x 4))

(define plus4 (lambda (x) (+ x 4)))
```

An operator can be a call expression too:

```
((lambda (x y z) (+ x y (square z))) 1 2 3)   ▶  12
```

Evaluates to the $x+y+z^2$ procedure

3. 以及作为一个编程语言拥有的多个输出和多个操作，以及命名操作

Cond & Begin

The cond special form that behaves like if-elif-else statements in Python

```
if x > 10:
    print('big')
elif x > 5:
    print('medium')
else:
    print('small')
```

```
(cond ((> x 10) (print 'big))
      ((> x 5)  (print 'medium))
      (else     (print 'small)))
```

```
(print
  (cond ((> x 10) 'big)
        ((> x 5)  'medium)
        (else     'small)))
```

The begin special form combines multiple expressions into one expression

```
if x > 10:
    print('big')
    print('guy')
else:
    print('small')
    print('fry')
```

```
(cond ((> x 10) (begin (print 'big)   (print 'guy)))
      (else     (begin (print 'small) (print 'fry))))
```

```
(if (> x 10) (begin
                (print 'big)
                (print 'guy))
             (begin
                (print 'small)
                (print 'fry)))
```

Let Expressions

The let special form binds symbols to values temporarily; just for one expression

```
a = 3
b = 2 + 2
c = math.sqrt(a * a + b * b)
a and b are still bound down here
```

```
(define c (let ((a 3)
                (b (+ 2 2)))
            (sqrt (+ (* a a) (* b b)))))
a and b are not bound down here
```

4. 再看一下循环

Write the procedure `repeat`, which takes in a procedure `f` and a number `n`, and outputs a new procedure. This new procedure takes in a number `x` and returns the result of calling `f` on `x` a total of `n` times. For example:

```
scm> (define (square x) (* x x))
square
scm> ((repeat square 2) 5) ; (square (square 5))
625
scm> ((repeat square 3) 3) ; (square (square (square 3)))
6561
scm> ((repeat square 1) 7) ; (square 7)
49
```

Hint: The `composed` function you wrote in the previous problem might be useful.

```
(define (repeat f n)
  ; note: this relies on `composed` being implemented correctly
  (if (< n 1)
    (lambda (x) x)
    (composed f (repeat f (- n 1))))
)
```

需要注意的是，此时看似这样写需要多个参数，实际上自己在迭代过程中，传一次参数就够了，相当于定义了一个重复n遍f的复合函数
其实一个很重要的提示点就是，最后传出compose传出的就是一个函数，因此就是函数套函数，可以这样简单理解

5. 一段很值得玩味的代码

```
(define (square n) (* n n))
(define (pow base exp)
(cond
((= exp 0)  1)
((even? exp) (square (pow base (/ exp 2))))
(else       (* base (pow base (- exp 1))))))
```

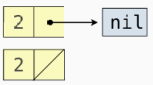
6. 神的编程之列表

Scheme Lists

In the late 1950s, computer scientists used confusing names

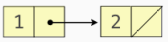
- **cons:** Two-argument procedure that creates a linked list
- **car:** Procedure that returns the first element of a list
- **cdr:** Procedure that returns the rest of a list
- **nil:** The empty list

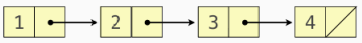
(cons 2 nil)



Important! Scheme lists are written in parentheses with elements separated by spaces

```
> (cons 1 (cons 2 nil))
(1 2)
> (define x (cons 1 (cons 2 nil))
> x
(1 2)
> (car x)
1
> (cdr x)
(2)
> (cons 1 (cons 2 (cons 3 (cons 4 nil))))
(1 2 3 4)
```





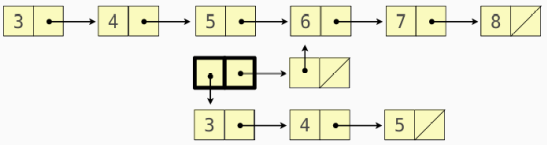
(Demo)

这是几个几个内置的函数

Recursive Construction

To build a list one element at a time, use **cons**
To build a list with a fixed length, use **list**

```
;;; Return a list of two lists; the first n elements of s and the rest
;;; scm> (split (list 3 4 5 6 7 8) 3)
;;; ((3 4 5) (6 7 8))
(define (split s n)
  ; The first n elements of s
  (define (prefix s n)
    (if (zero? n) nil (cons (car s) (prefix (cdr s) (- n 1)))))
  ; The elements after the first n
  (define (suffix s n)
    (if (zero? n) s (suffix (cdr s) (- n 1))))
  (list (prefix s n) (suffix s n)))
```



以及这个分割操作实在是太帅了

7. 神的编程之符号表示

Symbolic Programming

Symbols normally refer to values; how do we refer to symbols?

```
> (define a 1)
> (define b 2)
> (list a b)
(1 2)
```

No sign of “a” and “b” in the resulting value

Quotation is used to refer to symbols directly in Lisp.

```
> (list 'a 'b)
(a b)
> (list 'a b)
(a 2)
```

Short for (quote a), (quote b): Special form to indicate that the expression itself is the value.

Quotation can also be applied to combinations to form lists.

```
> '(a b c)
(a b c)
> (car '(a b c))
a
> (cdr '(a b c))
(b c)
```

(Demo)

8. 其中的一些内置函数

Built-in List Processing Procedures

(**append s t**): list the elements of s and t; append can be called on more than 2 lists

(**map f s**): call a procedure f on each element of a list s and list the results

(**filter f s**): call a procedure f on each element of a list s and list the elements for which a true value is the result

(**apply f s**): call a procedure f with the elements of a list s as its arguments

1. `(append s t)`

功能：

将两个或多个列表 `s` 和 `t` 的元素连接起来，返回一个新的列表。

参数：

- `s`：第一个列表。
- `t`：第二个列表（可以是多个列表）。

返回值：

一个新列表，包含所有输入列表的元素，按顺序拼接。

示例：

```scheme
(append '(1 2 3) '(4 5 6)) ; 结果为 (1 2 3 4 5 6)
```

说明：

- `append` 可以接受任意数量的列表作为参数，例如：

```scheme
(append '(1 2) '(3 4) '(5 6)) ; 结果为 (1 2 3 4 5 6)
```

2. `(map f s)`

功能：

对列表 `s` 的每个元素应用函数 `f`，并将结果组成一个新的列表返回。

参数：

- `f`：一个过程（函数），用于处理列表中的每个元素。
- `s`：一个列表。

返回值：

一个新列表，其中每个元素是原列表中对应元素经过函数 `f` 处理后的结果。

示例：

```scheme
(define (square x) (* x x)) ; 定义一个平方函数
(map square '(1 2 3 4)) ; 结果为 (1 4 9 16)
```

说明：

- 如果有多个列表，`map` 可以同时处理多个列表，并将它们的对应元素传递给函数 `f`。例如：

```scheme
(define (add x y) (+ x y)) ; 定义一个加法函数
(map add '(1 2 3) '(4 5 6)) ; 结果为 (5 7 9)
```

---

3. `(filter f s)`

功能：

对列表 `s` 的每个元素应用函数 `f`，并筛选出那些使得函数 `f` 返回真值（非空或非 `#f`）的元素，组成一个新的列表返回。

参数：

- `f`：一个过程（函数），用于判断列表中的每个元素是否满足条件。
- `s`：一个列表。

返回值：

一个新列表，其中只包含原列表中满足条件的元素。

示例：

```scheme
(define (even? x) (= (remainder x 2) 0)) ; 定义一个判断偶数的函数
(filter even? '(1 2 3 4 5 6)) ; 结果为 (2 4 6)
```

说明：

- 函数 `f` 应该返回一个布尔值（`#t` 或 `#f`）。如果返回 `#t`，则对应的元素会被保留；否则会被过滤掉。

4. `(apply f s)`

功能：

将列表 `s` 的元素展开为参数，调用函数 `f`。

参数：

- `f`：一个过程（函数），接受多个参数。
- `s`：一个列表，其元素将作为参数传递给函数 `f`。

返回值：

函数 `f` 调用的结果。

示例：

```scheme
(apply + '(1 2 3 4)) ; 等价于 (+ 1 2 3 4)，结果为 10
```

说明：

- `apply` 的作用是将列表中的元素"解包"为函数的参数。例如：

```scheme
(define (sum a b c) (+ a b c))
(apply sum '(1 2 3)) ; 结果为 6
```

9. 有时候想夸夸自己还是有点实力的（笑）

Definition: A perfect square is `k*k` for some integer `k`.

Implement `fit`, which takes non-negative integers `total` and `n`. It returns whether there are `n` **different** positive perfect squares that sum to `total`.

Important: Don't use the Scheme interpreter to tell you whether you've implemented it correctly. Discuss! On the final exam, you won't have an interpreter.

```
; Return whether there are n perfect squares with no repeats that sum to total

    (define (fit total n)
        (define (f total n k)
            (if (and (= n 0) (= total 0))
                #t
            (if (< total (* k k))
                #f
            (or (f (- total (* k k ) n-1 (+ k 1)) (f total n (+ k 1))))
            )))
        (f total n 1))

    (expect (fit 10 2) #t)  ; 1*1 + 3*3
    (expect (fit 9 1)  #t)  ; 3*3
    (expect (fit 9 2)  #f)  ;
    (expect (fit 9 3)  #f)  ; 1*1 + 2*2 + 2*2 doesn't count because of repeated 2*2
    (expect (fit 25 1)  #t) ; 5*5
    (expect (fit 25 2)  #t) ; 3*3 + 4*4

```

这个题就是看是否需要的一个题

10. 一定分清这个是怎么进行构建的（还是感慨这个保存页的强大）

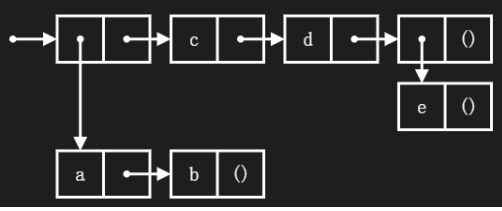
Untitled*

```
(define with-list
        (list
            (list 'a 'b) 'c 'd (list 'e)
        )
    )
    ; (draw with-list)  ; Uncomment this line to draw with-list

```

Untitled (Output)

```
scm> (draw with-list)
```

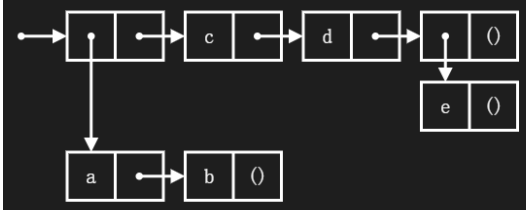


```
(define with-quote
        '(
            (a b) c d (e)
        )

    )
    ; (draw with-quote)  ; Uncomment this line to draw with-quote
```

Untitled (Output)

```
ith-quote
cm> (draw with-quote)
```



10. 来看这个题，学习一点东西

Implement a procedure called `ascending?`, which takes a list of numbers `s` and returns `True` if the numbers are in non-descending order, and `False` otherwise.

A list of numbers is non-descending if each element after the first is greater than or equal to the previous element. For example...

- `(1 2 3 3 4)` is non-descending.
- `(1 2 3 3 2)` is not.

Hint: The built-in `null?` procedure returns whether its argument is `nil`.

Note: The question mark in `ascending?` is just part of the procedure name and has no special meaning in terms of Scheme syntax. It is a common practice in Scheme to name procedures with a question mark at the end if it returns a boolean value.

```
(define (ascending? s)
  (if (or (null? s) (null? (cdr s)))
      #t
      (and (<= (car s) (car (cdr s))) (ascending? (cdr s))))
)
```

其中null? 是一个内置的判断函数；使用#t 来表示正确；cdr 和car 函数的使用。

11. 还有这个题，也是相当精妙

Write a procedure `my-filter`, which takes in a one-argument predicate function `pred` and a list `s`, and returns a new list containing only elements in list `s` that satisfy the predicate. The returned list should contain the elements in the same order that they appeared in the original list `s`.

For example, `(my-filter even? '(1 2 3 4 5))` should return `(2 4)` because only `2` and `4` are even.

> **Note:** You are **not allowed** to use the Scheme built-in `filter` function in this question - we are asking you to re-implement this!

```
(define (my-filter pred s)
  (cond ((null? s) '())
        ((pred (car s)) (cons (car s) (my-filter pred (cdr s))))
        (else (my-filter pred (cdr s))))
)
```

首先，我们必须认清，该类循环只能使用递归来写；齐次，必须写出终止条件；最后，构建一个新的列表，常用的套路。

12. fliter函数的巧用，进行筛选

Implement `no-repeats`, which takes a list of numbers `s`. It returns a list that has all of the unique elements of `s` in the order that they first appear, but no repeats.

For example, `(no-repeats (list 5 4 5 4 2 2))` evaluates to `(5 4 2)`.

> **Hint:** You may find it helpful to use `filter` with a `lambda` procedure to filter out repeats. To test if two numbers `a` and `b` are not equal, use `(not (= a b))`.

```
(define (no-repeats s)
  (if (null? s) s
    (cons (car s)
      (no-repeats (filter (lambda (x) (not (= (car s) x))) (cdr s))))))
)
```